JAVA CORE CONCEPTS

Strings in Java (Foundation Guide)

Object representation, immutability rules, memory equality, and foundational text problems

> Till now, you have learned variables to store single values, arrays to store multiple values, loops to process data, and conditionals to make decisions. Now comes a fundamental topic: **Strings — managing and manipulating text data in Java**.

1. What is a String & Why Does It Matter?

A **String** is a sequence of characters enclosed in double quotes.

```
String name = "Love";
String course = "Java DSA";
```

Real-World Applications: Strings are used everywhere across software development — usernames (`"Riya"`), course titles (`"DSA Bootcamp"`), feedback messages (`"Welcome to CodeHelp ONE"`), and email validations (`"user@gmail.com"`).

Creation Methods

- **Using String Literal (Recommended):** `String str = "Hello";` (Utilizes the String Constant Pool for memory efficiency).
- **Using new Keyword:** `String str = new String("Hello");` (Creates a new object instance explicitly in the Heap).

> ⚠️ **String is NOT a Primitive Type**
>
> Unlike `int` , `double` , or `char` , `String` is a reference object/class in Java. This grants it powerful built-in instance methods like `length()` , `charAt()` , and `equals()` .

2. Core String Mechanics

Indexing & Character Access

Strings are zero-indexed sequences. Accessing characters uses `charAt(index)` , and total length is retrieved via `length()` .

Index	0	1	2	3	4	5	6	7
Character	C	o	d	e	H	e	l	p

```
String str = "CodeHelp";
System.out.println(str.length());   // Outputs 8
System.out.println(str.charAt(0)); // Outputs 'C'
System.out.println(str.charAt(3)); // Outputs 'e'
```

Strings are Immutable (Crucial Concept)

Once created, a `String` object's internal character array **cannot be modified**. Methods that appear to alter a string actually return a brand-new `String` object.

```
String str = "Hello";
str.concat(" World");
System.out.println(str); // Still outputs "Hello"!

// Correct Reassignment Pattern:
str = str.concat(" World"); // Reassigns reference to new object -> "Hello World"
```

Comparing Strings (== vs .equals())

❌ Incorrect Comparison

```
if (str1 == str2) { ... }
```

Compares **memory reference addresses** in heap/pool. Returns `false` for different objects containing identical text.

✅ Correct Comparison

```
if (str1.equals(var2)) { ... }
```

Compares **actual sequence content** character by character. Returns `true` if contents match.

Console Input Methods

Method	Reads	Behavior
`sc.next()`	Single Word	Stops reading as soon as whitespace is reached.
`sc.nextLine()`	Full Line	Reads the entire line including spaces up to line break.

3. Basic String Problems

Problem 1 & 2: Traversal & Manual Length Counting

```
// Problem 1: Print Each Character
String str = "Code";
for (int i = 0; i < str.length(); i++) {
    System.out.println(str.charAt(i));
}

// Problem 2: Count Length Without length() Method
int count = 0;
for (int i = 0; i < str.length(); i++) {
    count++;
}
```

Problem 3: Count Vowels

```java
String str = "CodeHelp";
int vowelCount = 0;

for (int i = 0; i < str.length(); i++) {
    char ch = str.charAt(i);
    if (ch == 'a' || ch == 'e' || ch == 'i' || ch == 'o' || ch == 'u' ||
        ch == 'A' || ch == 'E' || ch == 'I' || ch == 'O' || ch == 'U') {
        vowelCount++;
    }
}
System.out.println("Vowels: " + vowelCount); // Vowels: 3 ('o', 'e', 'e')
```

Problem 4 & 5: Reverse String & Check Palindrome

```java
// Problem 4: Reverse String
String original = "Java";
String reversed = "";
for (int i = original.length() - 1; i >= 0; i--) {
    reversed += original.charAt(i);
}
System.out.println(reversed); // "avaJ"

// Problem 5: Check Palindrome
String pStr = "madam";
String pRev = "";
for (int i = pStr.length() - 1; i >= 0; i--) {
    pRev += pStr.charAt(i);
}

if (pStr.equals(pRev)) {
    System.out.println("Palindrome");
} else {
    System.out.println("Not Palindrome");
}
```

4. Frequently Used String Methods Summary

Method Header	Primary Function & Usage
`length()`	Returns the number of characters in the string.
`charAt(int index)`	Retrieves the character at the specified 0-based index.
`equals(String other)`	Compares actual content equality against another string.
`toLowerCase()`	Converts all characters to lowercase.
`toUpperCase()`	Converts all characters to uppercase.
`concat(String str)`	Appends text to the string and returns a new string instance.

⚠️ Common String Pitfalls

- **IndexOutOfBoundsException:** Calling `str.charAt(str.length())` instead of stopping at `str.length() - 1`.
- **Reference Comparison Trap:** Using `==` when checking string contents.
- **Immutability Assumption:** Expecting `str.toLowerCase()` to modify `str` in place without reassigning the return value.

💡 Interview & Algorithm Relevance

String fundamentals lead directly into advanced DSA topics:

- Sliding Window pattern matching (e.g., Anagram search)
- Hashing & Frequency Maps (e.g., Character counts, Isomorphic strings)
- Two-Pointer string reversing and palindrome validation
- Substring & Prefix Sum algorithms

📝 Homework & Practice Challenges

1. Count the total number of consonants in a string.
2. Convert a string to uppercase without using `toUpperCase()` (Hint: ASCII offset manipulation).
3. Find the frequency of a specific character in a given string.
4. Remove all spaces from a sentence string.
5. Check if a string contains only numeric digits.
6. Count the total number of words in a space-separated sentence.